AutoJudge:

Predicting Programming Problem Difficulty

A Machine Learning–Based System for Classifying Programming Problems

and Predicting Difficulty Scores Using Textual Information

Submitted by:

Gopal Datt Sharma (23115047)

Electrical Engineering III year

1. Introduction

Online coding platforms such as competitive programming websites and learning portals host a vast number of programming problems. These problems are commonly categorized into difficulty levels such as Easy, Medium, and Hard to help learners select appropriate challenges. However, difficulty labeling is often subjective and may vary across platforms and problem setters.

With the rapid growth of problem repositories, manual difficulty assignment becomes inefficient and inconsistent. Automated approaches to difficulty prediction can help standardize labeling and assist learners, educators, and platform administrators in organizing and recommending problems more effectively.

This project focuses on predicting the difficulty of programming problems using only textual information available in problem statements. The proposed system performs two tasks: (1) classification of problems into Easy, Medium, or Hard categories, and (2) prediction of a numerical difficulty score that reflects relative complexity. By leveraging natural language processing techniques and machine learning models, the system aims to provide scalable and consistent difficulty estimation.

The remainder of this report describes the dataset used, data preprocessing and feature extraction techniques, machine learning models employed for classification and regression, experimental results, and the implementation of a web-based interface for interactive predictions.

2. Dataset Description

The dataset used in this project consists of programming problems collected from various online coding platforms. Each problem is labeled with both a categorical difficulty level and a numerical difficulty score, making the dataset suitable for supervised learning tasks.

Each data instance in the dataset contains the following attributes:

- Title of the programming problem

- Problem description

- Input description

- Output description

- Sample input/output examples

- Difficulty class label (Easy, Medium, Hard)

- Numerical difficulty score

	title	description	input_description	output_description	sample_io	problem_class	problem_score
0	Uuu	Unununiun (Uuu) was the name of the chemical\n...	The input consists of one line with two intege...	The output consists of M lines where the i...	[{'input': '7 10', 'output': '1 2 2 3 1 3 3 4 ...	hard	9.7
1	House Building	A number of eccentrics from central New York h...	The input consists of 10 test cases, which a...	Print K lines with\n the positions of the...	[{'input': '0 2 3 2 50 60 50 30 50 40', 'outpu...	hard	9.7
2	Mario or Luigi	Mario and Luigi are playing a game where they ...			[{'input': '', 'output': ''}]	hard	9.6
3	The Wire Ghost	Žofka is bending a copper wire. She starts wit...	The first line contains two integers L and $...	The output consists of a single line consistin...	[{'input': '4 3 3 C 2 C 1 C', 'output': 'GHOST...	hard	9.6
4	Barking Up The Wrong Tree	Your dog Spot is let loose in the park. Well, ...	The first line of input consists of two intege...	Write a single line containing the length need...	[{'input': '2 0 10 0 10 10', 'output': '14.14'...	hard	9.6

The dataset contains a total of 4,112 programming problems and is provided in JSONL format. The dataset link mentioned in the project description was used as a reference, and an equivalent labeled dataset was used for training and evaluation.

The presence of both categorical and numerical labels enables the implementation of both classification and regression models within the same framework.

3. Data Handling

Data handling was performed to ensure the quality and reliability of the dataset before applying any machine learning models. Since the dataset was collected from multiple sources, it was necessary to verify its consistency and correctness.

The dataset was first checked for duplicate records, and duplicate entries were removed to avoid biased learning. Missing values in textual fields such as problem descriptions, input descriptions, and output descriptions were handled by replacing them with empty strings to maintain uniformity across samples.

Additionally, the consistency of difficulty class labels and numerical difficulty scores was verified to ensure that all records followed the same labeling scheme. No invalid or out-of-range score values were observed.

These data handling steps ensured that the dataset was clean, well-structured, and suitable for subsequent preprocessing, feature extraction, and model training.

4. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to gain insights into the characteristics of the dataset before model development. Visual analysis was used to understand class distribution, textual complexity, and difficulty score patterns.

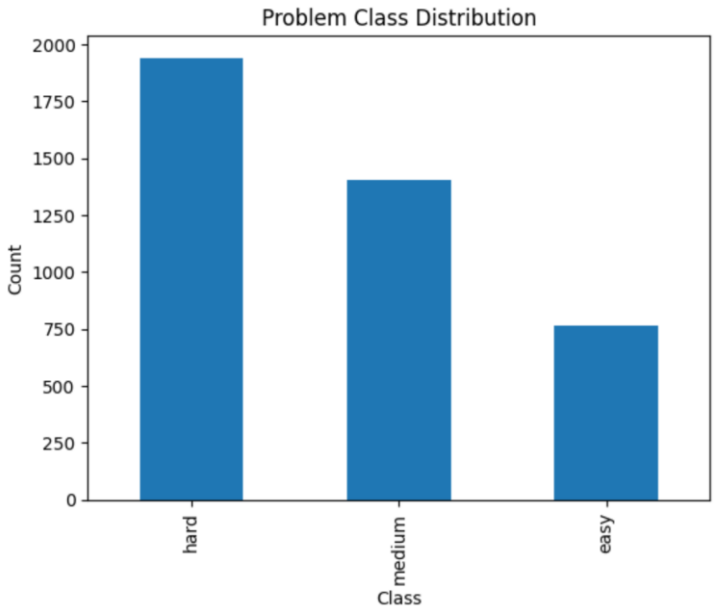


Figure 1: Problem Class Distribution

Figure 1 presents the distribution of programming problems across difficulty classes. The plot shows that the Hard class contains the largest number of problems, followed by Medium and Easy classes. This indicates a moderate class imbalance in the dataset, which motivated careful evaluation of class-wise performance during classification.

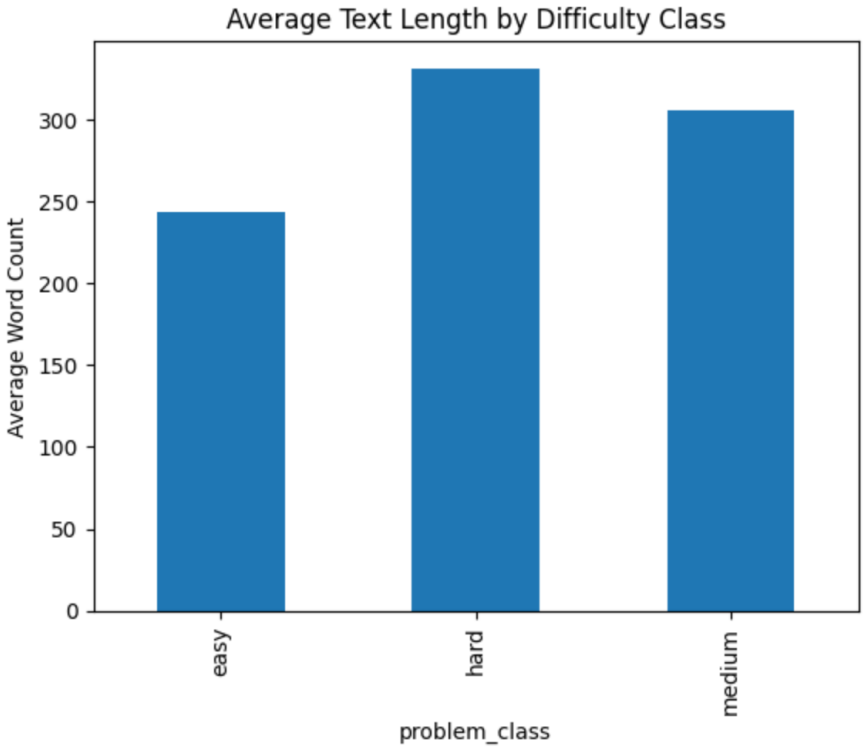


Figure 2: Average Text Length by Difficulty Class

Figure 2 illustrates the average text length of problem statements for each difficulty class. The analysis reveals that Hard problems tend to have longer textual descriptions on average compared to Medium and Easy problems. This suggests that textual complexity, such as longer descriptions and more detailed constraints, may be correlated with higher difficulty levels.

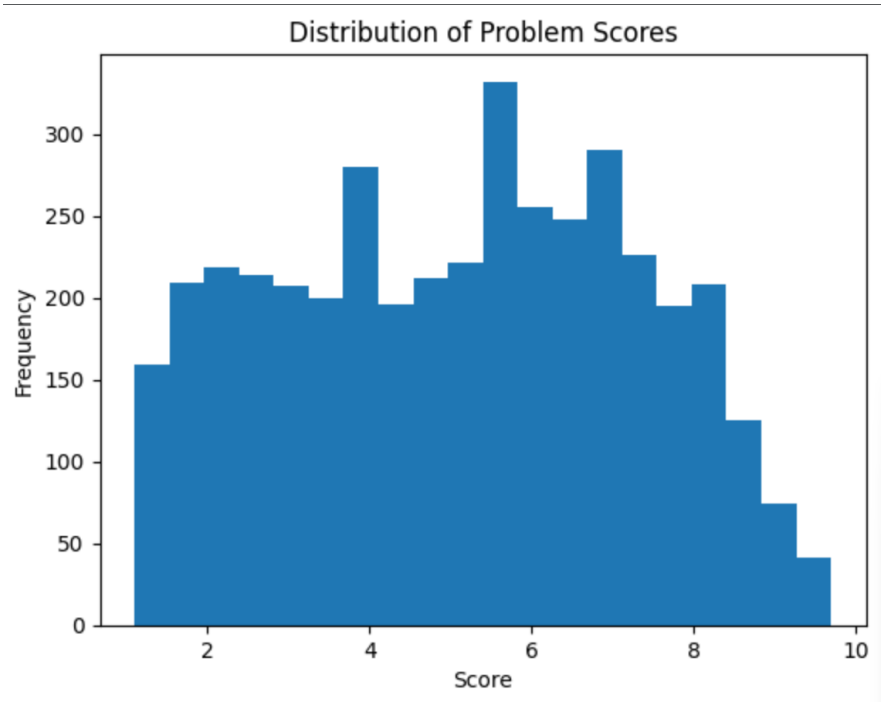


Figure 3: Distribution of Problem Scores

Figure 3 shows the distribution of numerical difficulty scores across all programming problems. The scores span a broad range, indicating significant variation in problem complexity. Most scores are concentrated around mid-range values, supporting the use of a regression model to capture continuous variations in difficulty rather than relying solely on discrete class labels.

These EDA observations confirm that textual features contain meaningful signals related to problem difficulty and justify the use of natural language processing techniques for both classification and regression tasks.

5. Data Preprocessing and Feature Engineering

The textual fields, including problem description, input description, and output description, were first normalized by converting text to lowercase and removing unnecessary whitespace. These fields were then combined into a single textual representation for each programming problem. Combining multiple textual components ensured that all relevant information contributed to the difficulty prediction.

	clean_text
0	uuu unununium (uuu) was the name of the chemic...
1	house building a number of eccentrics from cen...
2	mario or luigi mario and luigi are playing a g...
3	the wire ghost žofka is bending a copper wire....
4	barking up the wrong tree your dog spot is let...

For feature extraction, the Term Frequency–Inverse Document Frequency (TF-IDF) technique was used to convert text into numerical feature vectors. TF-IDF assigns higher importance to informative words while reducing the impact of frequently occurring but less meaningful terms. This representation is well suited for traditional machine learning algorithms operating on textual data.

The processed feature vectors were then split into training and testing sets to enable objective evaluation of model performance. The same TF-IDF representation was used consistently across both classification and regression tasks to ensure comparability and reproducibility.

These preprocessing and feature engineering steps transformed raw textual data into structured numerical features suitable for machine learning model training.

6. Classification Models

The classification task aims to predict the difficulty level of programming problems as Easy, Medium, or Hard based on textual features.

Several machine learning classifiers were explored during experimentation, including Logistic Regression, Support Vector Machine (SVM), Random Forest, XGBoost, LightGBM, and CatBoost. All models were trained using TF-IDF–based textual feature representations.

Model performance was evaluated using classification accuracy and confusion matrices. Among the evaluated models, the Random Forest classifier achieved the highest accuracy and demonstrated better class-wise performance, particularly for the Hard category.

Accurate identification of Hard problems was considered especially important, as misclassifying complex problems as Easy could negatively impact learning and problem selection. Confusion matrix analysis showed that the Random Forest model reduced such extreme misclassifications and produced stronger diagonal dominance compared to other classifiers.

Based on these observations, the Random Forest classifier was selected as the final model for difficulty classification.

7. Regression Models

The regression task focuses on predicting a numerical difficulty score for each programming problem based on textual features.

Multiple regression models were evaluated during experimentation, including Linear Regression, Random Forest Regressor, Gradient Boosting Regressor, XGBoost, LightGBM, and CatBoost Regressor. All models were trained using the same TF-IDF–based feature representation to ensure fair comparison.

Regression performance was evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). These metrics quantify the average prediction error and penalize larger deviations more heavily.

Among the evaluated models, the CatBoost Regressor achieved the lowest MAE and RMSE values, indicating more accurate difficulty score predictions. Its ability to capture non-linear relationships in the data made it well suited for this task.

Based on quantitative evaluation results, the CatBoost Regressor was selected as the final model for difficulty score prediction.

8. Experimental Results and Evaluation

This section presents the experimental results obtained from the classification and regression models, along with their evaluation using standard performance metrics.

- **Classification Model Comparison**

Table 1 presents the performance comparison of different classification models evaluated for predicting the difficulty class of programming problems. Classification accuracy was used as the primary evaluation metric.

Model	Accuracy
Logistic Regression	~0.47
Support Vector Machine	~0.48
Random Forest	~0.53
XGBoost	~0.51
LightGBM	~0.49
CatBoost	~0.51

Among the evaluated classifiers, the **Random Forest model** achieved the highest accuracy of **53%**. In addition to improved accuracy, confusion matrix analysis showed better identification of Hard problems, which motivated its selection as the final classification model.

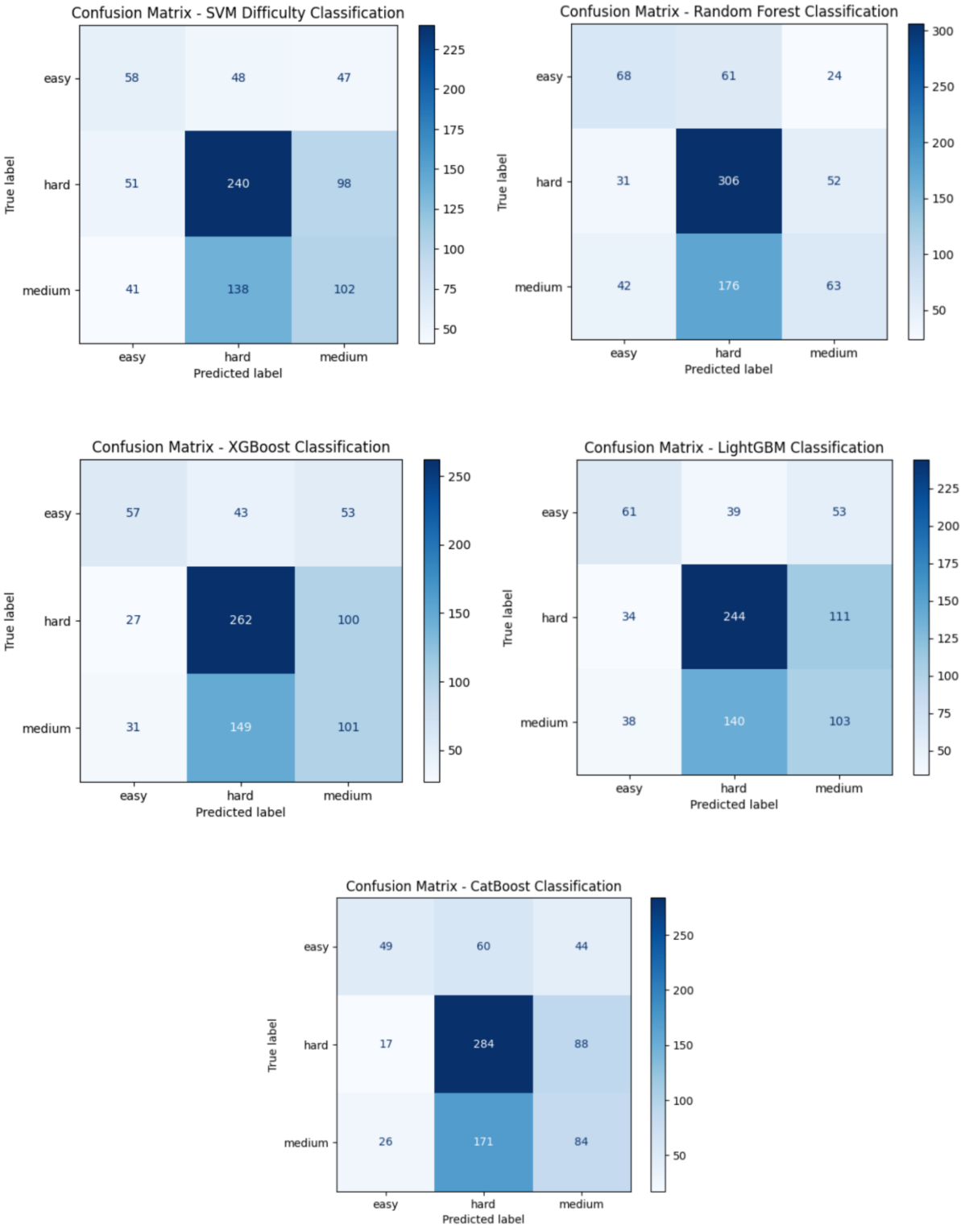
- **Confusion Matrix Analysis**

The confusion matrices for Support Vector Machine (SVM), XGBoost, LightGBM, and CatBoost classifiers are presented below. These matrices provide insight into how different models handle misclassification across Easy, Medium, and Hard difficulty levels.

Overall, it was observed that several models struggled to clearly separate adjacent difficulty classes, particularly Medium and Hard. Some models also showed a higher tendency to misclassify Hard problems as Medium or Easy. Compared to these models, the **Random Forest classifier**

demonstrated stronger diagonal dominance and fewer extreme misclassifications, especially for the Hard class.

These observations further support the selection of Random Forest as the final classification model for this project.



- **Regression Model Comparison**

Table 2 presents the performance comparison of regression models evaluated for predicting numerical difficulty scores. Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) were used as evaluation metrics.

Model	MAE	RMSE
Linear Regression	~3.40	~4.30
Random Forest Regressor	~1.71	~2.04
Gradient Boosting	~1.71	~2.04
XGBoost	~1.74	~2.09
LightGBM	~1.72	~2.10
CatBoost	~1.68	~2.03

The **CatBoost Regressor** achieved the lowest MAE of **1.68** and lowest RMSE of **2.03** among all evaluated models, indicating more accurate difficulty score prediction. Based on these results, **CatBoost** was selected as the final regression model.

```
CatBoost MAE: 1.6871368821169963
CatBoost RMSE: 2.0357712595520456
```

Overall, the experimental results demonstrate that the proposed approach provides meaningful difficulty predictions based solely on textual information.

9. Web Interface and Sample Predictions

A simple web-based interface was developed using **Streamlit** to enable interactive difficulty prediction for programming problems. The interface allows users to input textual information and obtain real-time predictions from the trained models.

The web interface consists of three input text fields:

- Problem Description

- Input Description

- Output Description

After entering the required information, users can submit the input to receive two outputs:

- Predicted difficulty class (Easy, Medium, or Hard)

- Predicted numerical difficulty score

The web interface loads the saved Random Forest classification model and CatBoost regression model, along with the TF-IDF vectorizer, to generate predictions. All computations are performed locally, and no external hosting is required.

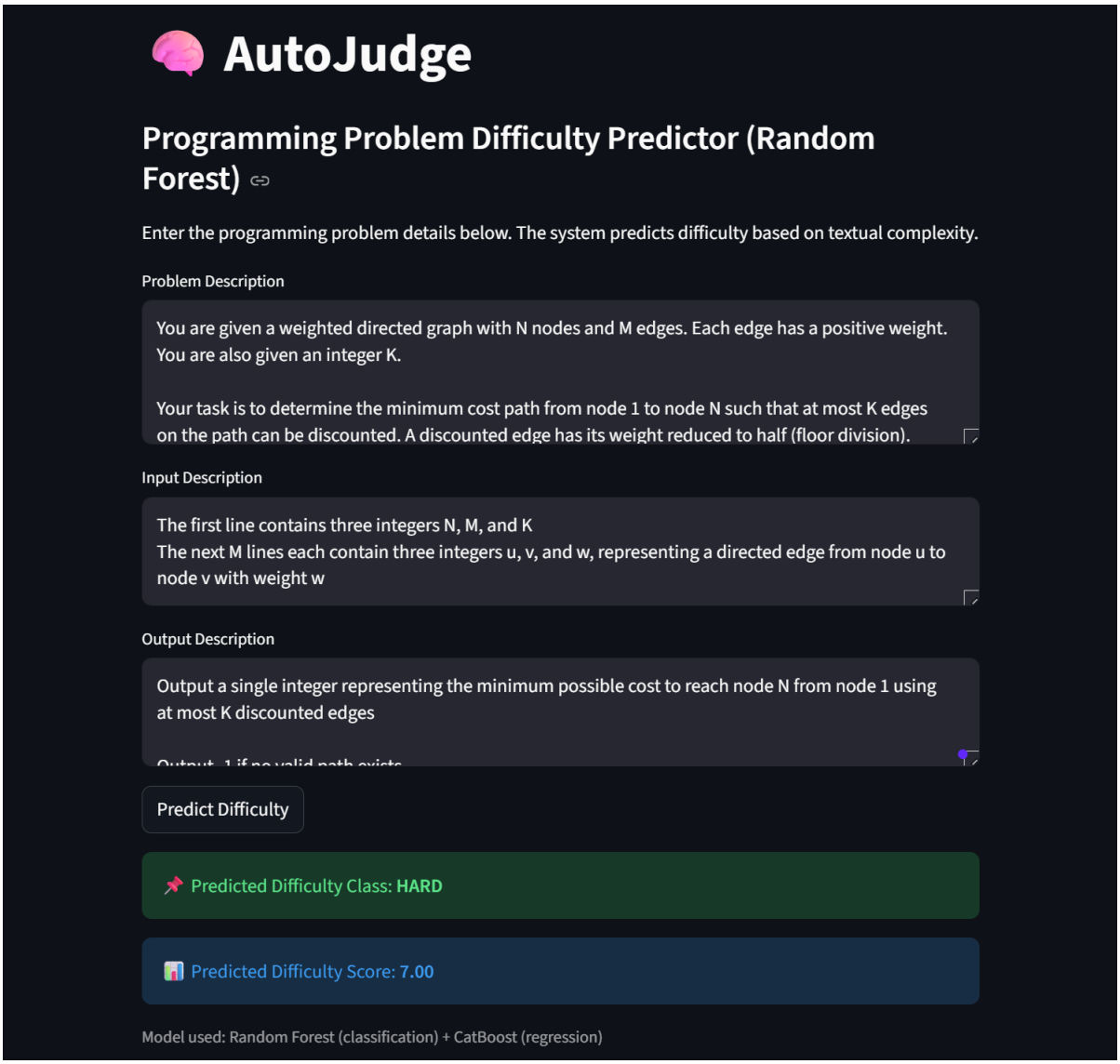


Figure 6 shows a screenshot of the web interface with sample input and predicted outputs.

The displayed predictions demonstrate the practical usability of the system for estimating problem difficulty based solely on textual descriptions.

10. Conclusion and Future Work

This project presented AutoJudge, a machine learning–based system for predicting the difficulty of programming problems using only textual information. The system was designed to perform both difficulty classification (Easy, Medium, Hard) and numerical difficulty score prediction.

Through exploratory data analysis, meaningful relationships between textual complexity and problem difficulty were observed. Text preprocessing and TF-IDF feature extraction were applied to transform problem statements into numerical representations suitable for machine learning models. Multiple classification and regression models were evaluated, and Random Forest and CatBoost were selected as the final models based on quantitative performance metrics and qualitative analysis.

The integration of trained models into a web-based interface demonstrated the practical usability of the system, allowing users to obtain real-time difficulty predictions from problem descriptions.

Future work may include incorporating additional features such as code-based metrics, constraint analysis, or problem tags to improve prediction accuracy. Deep learning approaches such as transformer-based language models could also be explored to capture deeper semantic information from problem statements. Expanding the dataset and addressing class imbalance through advanced techniques may further enhance model performance.